



VLSI PHYSICAL DESIGN INTERNSHIP FINAL PROJECT

NAME: V Pardhu Sai Harsha

REG NO: 24BEC1091

EMAIL: pardhusaiharshav@gmail.com

UNIVERSITY NAME: VELLORE INSTITUTE OF
TECHNOLOGY, CHENNAI

LOCATION: CHENNAI

Contents

S.No	Content	Page Number
1	Abstract	03
2	Introduction	04
3	Serial Peripheral Interface (SPI)	06
4	Invoke the tool	07
5	Synthesis	08
6	Placement	11
7	Static Timing Analysis	12
8	Routing	13
9	DRC	14
10	LVS	17
11	GDS	19
12	Results and Discussion (Project results)	21
13	Conclusion	26
14	Learning Outcomes	27

Abstract :

The semiconductor industry plays a very important role in today's world because almost every electronic device we use, such as smartphones, laptops, smartwatches, automobiles, medical equipment, and communication systems, depends on integrated circuits (ICs). As technology continues to advance, there is an increasing demand for smaller, faster, and more power-efficient chips. To meet these requirements, engineers use Very Large Scale Integration (VLSI) technology, which allows millions and even billions of transistors to be integrated onto a single silicon chip. VLSI design has become one of the most important fields in electronics and semiconductor engineering because it enables the development of complex digital systems with high performance and reliability.

Designing an integrated circuit is not limited to writing hardware description code. After the digital functionality of a design is created, it must go through several stages before it can be manufactured as a physical chip. This process is known as the ASIC (Application Specific Integrated Circuit) Physical Design Flow. The physical design flow converts a logical circuit description into a physical layout that can be fabricated on silicon. It involves multiple steps such as synthesis, floorplanning, placement, clock tree implementation, routing, timing analysis, physical verification, and final layout generation. Each stage is important because even a small mistake can affect the performance or functionality of the chip.

This internship project focuses on understanding and implementing the complete ASIC physical design flow using an open-source VLSI design environment called Qflow. The design selected for this project is the Serial Peripheral Interface (SPI), which is a widely used communication protocol in embedded systems and digital electronics. SPI is commonly used for communication between microcontrollers and peripheral devices such as sensors, memory modules, displays, and other integrated circuits. Due to its simplicity and high-speed communication capability, SPI is one of the most popular serial communication protocols in modern electronic systems.

The SPI design was first developed using Verilog Hardware Description Language (HDL). Verilog is used to describe the behavior and structure of digital circuits. After verifying the RTL (Register Transfer Level) design, the Verilog code was processed through the ASIC design flow using Qflow. The first stage involved logic synthesis, where the RTL description was converted into a gate-level netlist using standard cell libraries. The synthesized netlist was then used for physical implementation.

Following synthesis, the design underwent placement and routing, where logic gates were physically arranged on the chip and electrical connections were created between them. Proper placement and routing are essential for achieving good timing performance and minimizing delays. After routing, Static Timing Analysis (STA) was performed to ensure that all timing requirements were satisfied and to estimate the maximum operating frequency of the design.

To verify the correctness of the generated layout, Design Rule Checking (DRC) and Layout Versus Schematic (LVS) verification were carried out. DRC ensures that the physical layout follows all manufacturing rules specified by the semiconductor technology, while LVS

verifies that the final layout matches the original circuit design. These verification steps are critical because they help identify errors before fabrication, reducing the risk of costly manufacturing failures.

Finally, the completed layout was exported as a GDSII (Graphic Data System II) file, which is the industry-standard format used for chip fabrication. The results obtained from the project demonstrated the successful implementation of the SPI design through the complete ASIC physical design flow. The final layout passed physical verification with zero DRC violations and zero LVS mismatches, indicating that the design was both manufacturable and functionally correct. Additionally, Static Timing Analysis confirmed the timing performance of the design and provided insights into its maximum achievable operating frequency.

Overall, this project provided valuable hands-on experience in backend VLSI design and helped in understanding how a digital design is transformed into a manufacturable integrated circuit. It also highlighted the importance of verification, timing optimization, and physical implementation in the semiconductor design process. Through this internship, practical knowledge of industry-standard ASIC design methodologies and physical design concepts was gained, strengthening the foundation for future work in the VLSI domain.

Introduction :

In today's world, electronic devices have become an important part of our daily lives. We use smartphones, laptops, smartwatches, televisions, and many other digital devices every day. As technology continues to improve, people expect these devices to become faster, smaller, more powerful, and energy-efficient. To meet these growing demands, the semiconductor industry continuously develops advanced integrated circuits (ICs) that can perform complex operations while occupying less space and consuming less power.

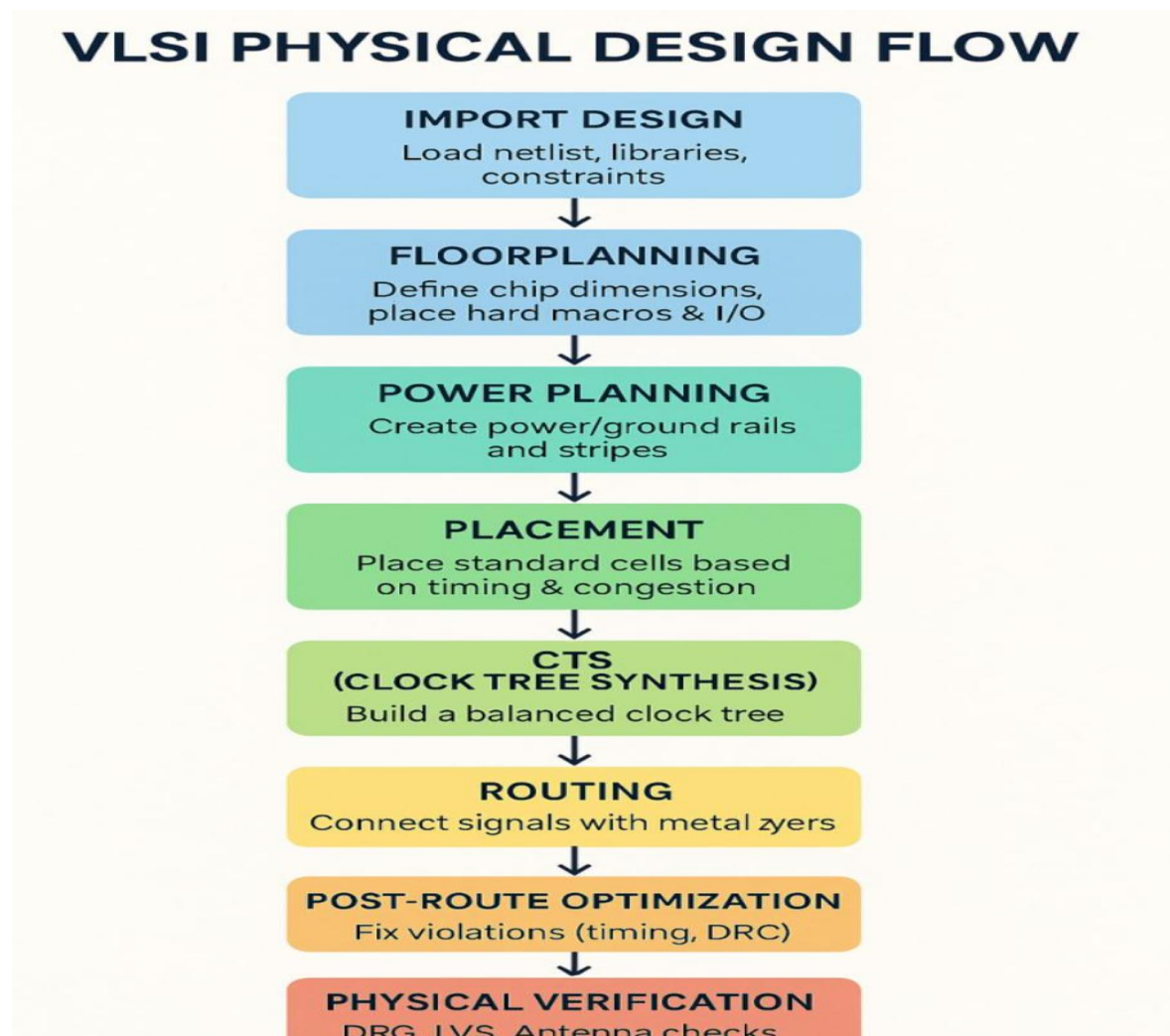
One of the key technologies that makes this possible is Very Large Scale Integration (VLSI). VLSI technology allows millions of transistors and electronic components to be integrated onto a single silicon chip. This has made it possible to design highly sophisticated systems such as processors, memory devices, communication modules, and artificial intelligence hardware. As applications like the Internet of Things (IoT), embedded systems, cloud computing, and machine learning continue to grow, the importance of VLSI design has increased significantly.

Designing an integrated circuit is not simply about writing code. A digital design must go through several stages before it can be manufactured as a physical chip. This entire process is known as the ASIC (Application Specific Integrated Circuit) design flow. The design flow begins with understanding the required functionality of the circuit and describing it using Hardware Description Languages (HDLs) such as Verilog. The design is then verified to ensure that it performs the intended operation correctly. After verification, it is converted into a gate-level representation through synthesis.

The ASIC design flow is generally divided into two major parts: Front-End Design and Back-End Physical Design. In the front-end stage, the functionality of the circuit is designed and

verified. This includes writing RTL code, simulation, debugging, and synthesis. Once the design is functionally correct, it moves to the back-end stage, where the physical implementation of the circuit is carried out.

The physical design stage is one of the most important phases in VLSI design because it transforms the synthesized netlist into an actual chip layout. Several steps are involved in this process, including floor planning, power planning, placement, clock tree synthesis, routing, timing analysis, and physical verification. The quality of these steps directly affects the final chip's performance, power consumption, area utilization, and reliability. Therefore, engineers must carefully optimize the design at every stage to achieve the best possible results.



In this internship project, the complete ASIC physical design flow is studied and implemented using Qflow, an open-source VLSI design automation tool. Qflow provides an easy-to-use environment that combines different open-source Electronic Design Automation (EDA) tools required for ASIC implementation. It enables users to perform synthesis, placement, routing, timing analysis, and layout generation while gaining practical exposure to real-world chip design processes.

The project titled "SPI Using Qflow" focuses on implementing the physical design flow for a Serial Peripheral Interface (SPI) module. SPI is a widely used communication protocol that

enables data transfer between microcontrollers and peripheral devices such as sensors, displays, memory chips, and communication modules. Due to its simple architecture and high-speed communication capability, SPI is commonly used in many embedded and digital systems.

As part of this project, the SPI design written in Verilog is taken through different stages of the ASIC design flow. The design is first synthesized into a gate-level netlist and then processed through floor planning, power planning, placement, routing, Static Timing Analysis (STA), Design Rule Check (DRC), Layout Versus Schematic (LVS) verification, and final GDS generation. Each stage is studied in detail to understand its purpose and contribution to the overall chip implementation process.

Working on this project provided practical exposure to backend VLSI design concepts and helped in understanding how a digital circuit is converted into a physical layout that can be manufactured on silicon. It also provided valuable knowledge about timing analysis, physical verification, and design optimization techniques that are widely used in the semiconductor industry.

Overall, this project helped in developing a better understanding of the complete ASIC physical design flow and the role of each stage in chip implementation. The experience gained through this internship strengthened both theoretical knowledge and practical skills in VLSI design, making it a valuable learning experience for pursuing a future career in the semiconductor industry.

Serial Peripheral Interface (SPI) :

The Serial Peripheral Interface (SPI) is a widely used synchronous serial communication protocol developed by Motorola for high-speed data transfer between a master device and one or more slave devices. It is commonly used in embedded systems to communicate with peripherals such as sensors, memory devices, displays, ADCs, DACs, and wireless modules.

SPI follows a master-slave architecture, where the master controls communication by generating the clock signal, selecting the slave device, and managing data transfer. Since SPI is a synchronous protocol, both devices use a common clock signal, enabling fast and reliable communication.

SPI supports full-duplex communication, allowing the master and slave to transmit and receive data simultaneously. Data is transferred serially through four main signals:

- SCLK (Serial Clock): Clock signal generated by the master for synchronization.
- MOSI (Master Out Slave In): Transfers data from the master to the slave.
- MISO (Master In Slave Out): Transfers data from the slave to the master.
- CS/SS (Chip Select/Slave Select): Selects the target slave device for communication.

SPI Communication Process

Communication begins when the master activates the Chip Select (CS) line of the desired slave. The master then generates clock pulses on SCLK while transmitting data through MOSI and simultaneously receiving data through MISO. After the required data transfer is complete, the master deactivates the CS signal, ending the communication.

Advantages of SPI

- High-speed data transfer
- Simple hardware implementation
- Full-duplex communication
- Low communication overhead
- Supports multiple slave devices
- Reliable clock-based synchronization

Applications of SPI

SPI is widely used in:

- Flash memory and EEPROM
- Sensors and sensor modules
- LCD/OLED displays
- ADCs and DACs
- Wireless communication modules
- Embedded and IoT systems
- Industrial automation and robotics

SPI in This Project

In this internship project, an SPI module is implemented to demonstrate the complete ASIC physical design flow using the Qflow open-source VLSI design environment. The SPI design is described in Verilog HDL and passes through synthesis, floorplanning, power planning, placement, routing, timing analysis, and physical verification (DRC and LVS). Finally, a GDS layout is generated, demonstrating the transformation of an RTL design into a manufacturable ASIC layout while providing practical experience in backend VLSI design.

Invoke the tool :

Qflow Tool Overview :

Qflow is an open-source VLSI design automation tool used to implement and study the complete ASIC physical design flow. In the semiconductor industry, Electronic Design

Automation (EDA) tools are used to convert a digital design into a physical chip layout. Qflow provides an integrated environment that allows users to perform different stages of the ASIC design process using open-source tools.

Purpose of Qflow :

The main purpose of Qflow is to demonstrate how a digital design described in Verilog is transformed into a physical layout. It helps users understand important backend VLSI design stages such as synthesis, placement, routing, and layout generation. Qflow also allows designers to analyze design parameters such as area, timing, and routing efficiency.

Qflow Design Flow :

The design flow in Qflow begins with a Verilog RTL design. The RTL code is first synthesized into a gate-level netlist and then processed through placement and routing stages. Finally, a physical layout is generated and can be viewed and analyzed for verification.

Major Tools Used in Qflow :

Yosys :

Yosys is the synthesis tool used to convert the Verilog RTL design into a gate-level netlist using standard cells.

Graywolf :

Graywolf is the placement tool that positions standard cells within the design area to optimize chip utilization.

Qrouter :

Qrouter is responsible for routing and creating physical connections between the placed cells according to the netlist.

Magic :

Magic is used to view, analyse, and verify the final physical layout generated after routing.

Qflow in This Project :

In this project, Qflow is used to implement the SPI design through the complete ASIC physical design flow. The SPI RTL code is synthesised, placed, routed, and converted into a physical layout. This process provides practical understanding of backend VLSI design and demonstrates how a digital circuit is transformed into a manufacturable chip layout.

Synthesis :

Logic synthesis is one of the most important stages in the ASIC design flow. It is the process of converting a digital design written in a Hardware Description Language (HDL), such as Verilog, into a gate-level netlist composed of standard logic cells from a technology library.

The main objective of synthesis is to transform the RTL (Register Transfer Level) description into an optimised hardware implementation that can be physically realised on silicon. During this process, the synthesis tool analyses the design and converts it into basic digital components such as AND gates, OR gates, multiplexers, flip-flops, inverters, and buffers.

In addition to conversion, the synthesis tool performs several optimisation techniques to improve the overall quality of the design. These optimisations help reduce chip area, improve timing performance, and minimise power consumption. As a result, the synthesized design becomes more efficient and suitable for further physical implementation.

The logic synthesis process generally consists of three major steps:

RTL Analysis

In the first stage, the synthesis tool reads the Verilog source code and checks the design for syntax correctness, module hierarchy, signal declarations, and clock/reset definitions. The functionality described in the RTL code is analysed and interpreted as digital hardware.

Logic Optimization

After understanding the design, the synthesis tool optimises the logic by removing redundant circuitry, simplifying Boolean expressions, and reducing unnecessary hardware resources. This helps improve the efficiency of the design while maintaining its functionality.

Technology Mapping

In the final stage, the optimised logic is mapped to the standard cells available in the selected technology library. The resulting design is represented as a gate-level netlist consisting of actual cells that can later be placed and routed during physical design.

Logic Synthesis in the SPI Project

In this project, the SPI controller RTL design was developed in Verilog and synthesised using the Yosys synthesis tool integrated within the Qflow environment.

The primary input file used for synthesis was:

`spi_top.v`

This file contains the complete SPI controller implementation, including:

- Wishbone interface logic
- SPI clock generation circuitry
- Master-slave communication logic
- MOSI data transmission logic
- MISO data reception logic
- Slave select control logic

The synthesis process was executed using the following command:

```
qflow -T osu018 synthesise spi_top
```

The selected technology library for synthesis was:

```
osu018_stdcells
```

Using this library, the RTL design was converted into an optimised gate-level netlist containing technology-mapped standard cells.

Generated Output Files

The synthesis stage generated several important output files:

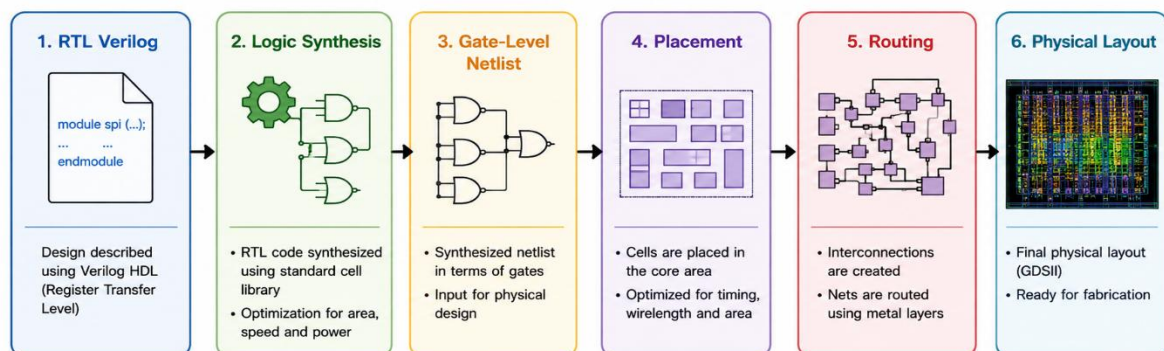
Output File	Description
spi_top.blif	Boolean logic representation of the synthesised design
spi_top_mapped.blif	Technology-mapped gate-level netlist
spi_top_synth.rtl.v	Synthesised Verilog representation

These files serve as the input for subsequent physical design stages.

Importance of Synthesis in ASIC Flow

Logic synthesis acts as a bridge between RTL design and physical implementation. It converts the behavioural description of the circuit into an optimised gate-level representation that can be physically implemented on silicon.

The overall design flow can be represented as:



A well-optimised synthesis result improves placement quality, routing efficiency, timing performance, and overall chip area utilisation.

Project Result

The SPI design was successfully synthesized using Qflow and Yosys. After resolving the required Verilog dependency files, the synthesis process completed without errors. The generated gate-level netlist was successfully produced and used as the input for the subsequent placement and routing stages of the ASIC physical design flow.

This stage confirmed that the SPI RTL design was functionally correct and ready for physical implementation.

Placement :

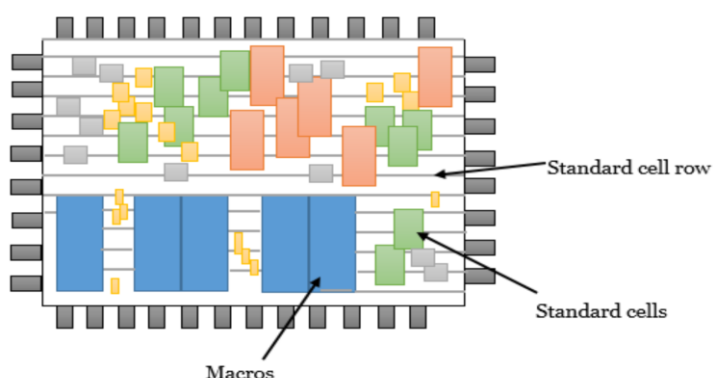
Placement is an important stage in the ASIC physical design flow, performed after logic synthesis. During synthesis, the RTL design is converted into a gate-level netlist, but the standard cells do not yet have physical locations. The placement stage assigns each standard cell to a valid position within the chip area.

The placement tool arranges cells by considering factors such as connectivity, timing requirements, routing resources, chip area, and power. Proper placement reduces wire length, minimizes signal delay, lowers power consumption, and reduces routing congestion, resulting in better overall design performance.

Placement is typically carried out in three phases: global placement, optimization, and legalization. Initially, cells are placed in approximate locations, then optimized to improve timing and reduce congestion, and finally legalized to ensure that cells do not overlap and occupy valid positions.

Objectives of Placement

- Minimise wire length between connected cells.
- Improve timing performance.
- Optimise chip area utilisation.
- Reduce routing congestion.
- Lower power consumption.
- Prepare the routing design.



Placement in the SPI Project

In this project, the synthesised SPI gate-level netlist generated by Yosys was used as the input for placement in the Qflow environment. The placement tool assigned physical locations to all standard cells while optimising connectivity and timing. Functional blocks such as control logic, clock generation, data transmission, and slave selection circuits were arranged efficiently to support improved routing performance.

Project Result

The SPI design was successfully placed using Qflow. All standard cells were assigned valid, non-overlapping physical locations, producing an optimised layout ready for the routing stage. This marked an important step in transforming the synthesised design into a manufacturable ASIC layout.

Static Timing Analysis (STA) :

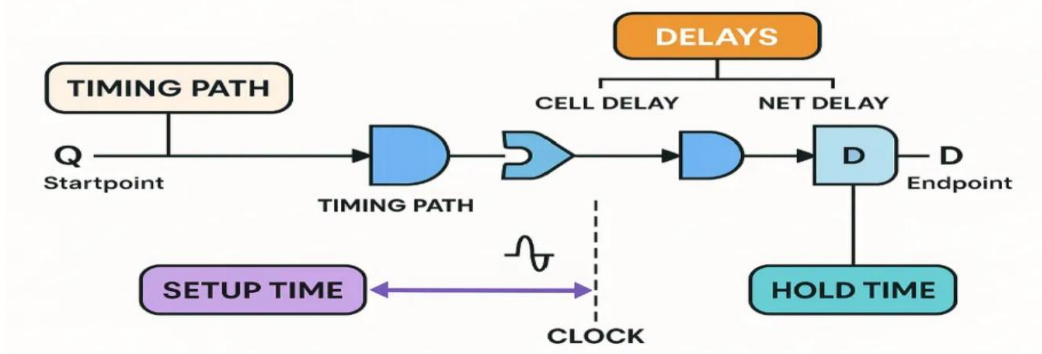
Static Timing Analysis (STA) is a key verification stage in the ASIC physical design flow used to evaluate whether a digital circuit meets its timing requirements. Unlike simulation-based verification, STA analyses the timing behaviour mathematically without applying input test vectors, making it faster and more efficient for large designs.

The main objective of STA is to verify that signals propagate through the circuit within the required timing constraints. It analyses timing paths, calculates delays, identifies critical paths, and ensures the design can operate correctly at the target clock frequency.

Important Timing Parameters

- **Setup Time:** Minimum time the input data must remain stable before the active clock edge.
- **Hold Time:** Minimum time the input data must remain stable after the active clock edge.
- **Propagation Delay:** Time taken for a signal to travel through logic gates and interconnections.
- **Clock Frequency:** The maximum operating frequency determined by the critical path delay.

KEY COMPONENTS OF STA



Importance of STA

STA helps to:

- Verify timing constraints.
- Detect setup and hold time violations.
- Identify critical paths.
- Estimate the maximum operating frequency.
- Improve circuit performance and reliability.

STA in the SPI Project

In this project, Static Timing Analysis was performed after placement to evaluate the timing performance of the SPI design. The analysis calculated delays along different signal paths, verified timing constraints, and identified critical paths affecting the operating frequency. The timing reports confirmed that the SPI controller met the required timing specifications.

Project Result

The SPI design successfully passed Static Timing Analysis. The generated timing reports showed no major timing violations, confirming that the synthesised and placed design satisfied the required timing constraints and was ready for the next stages of the ASIC physical design flow.

Routing :

Routing is an important stage in the ASIC physical design flow performed after placement. During placement, standard cells are assigned physical locations, but they are not electrically connected. The purpose of routing is to create the physical interconnections between these cells using metal layers, ensuring the circuit functions according to the design specifications.

Routing must establish all required signal paths while following the technology design rules. Efficient routing reduces signal delay, minimises congestion, improves reliability, and prepares the design for physical verification.

Stages of Routing

Global Routing:

In this stage, the routing tool determines approximate paths for signal connections and estimates routing resources while minimizing congestion.

Detailed Routing:

The routing tool creates the actual metal wires and vias to connect all cells, ensuring that all technology design rules are satisfied.

Objectives of Routing

- Complete all required signal connections.
- Minimize signal delay.
- Reduce routing congestion.
- Optimize routing resource utilization.
- Follow technology design rules.
- Prepare the design for physical verification.

Routing in the SPI Project

In this project, routing was performed after the successful placement of the SPI design using the Qflow environment. The routing tool created all required interconnections between the placed standard cells, including connections for the clock, control logic, data transmission, data reception, and slave selection circuits. The routing process minimized congestion while complying with the technology design rules.

Project Result

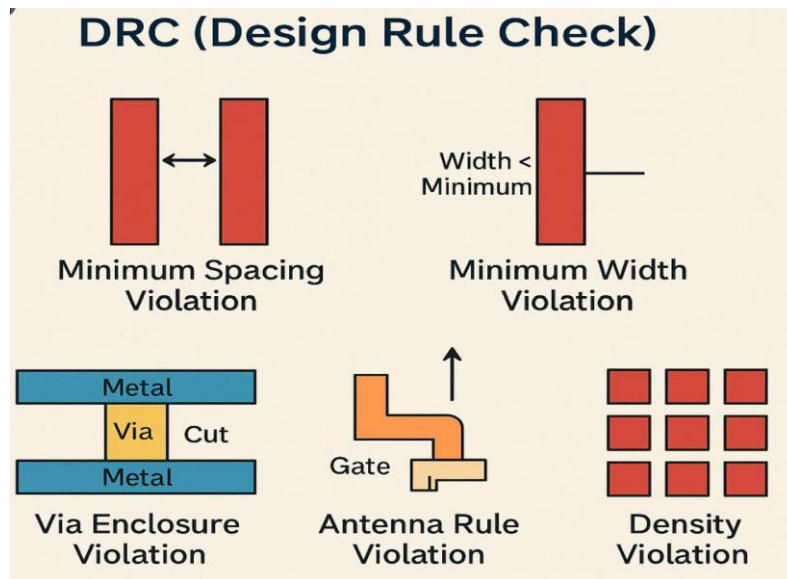
The SPI design was successfully routed using Qflow. All required signal connections were completed without major routing issues, producing a fully connected physical layout. The routed design was then ready for physical verification stages such as Design Rule Check (DRC), Layout Versus Schematic (LVS), and final GDS generation.

DRC (Design Rule Check) :

Design Rule Check (DRC) is an important verification stage in the ASIC physical design flow. It is performed after the routing stage to ensure that the generated layout follows all the manufacturing rules specified by the semiconductor technology. These rules are provided by the fabrication process and must be satisfied to ensure that the chip can be manufactured correctly without defects.

As modern integrated circuits contain millions of transistors packed into a very small area, even a minor layout mistake can lead to manufacturing problems or circuit failures. Therefore, DRC is used to identify and correct such issues before the design proceeds to fabrication.

The primary objective of DRC is to verify that every component and interconnection in the layout complies with the physical constraints of the selected technology. If any rule is violated, the DRC tool reports the error so that it can be corrected before moving to the next stage.



Parameters Checked During DRC

The DRC process examines several important layout parameters, including:

Minimum Spacing

The tool checks whether sufficient spacing is maintained between adjacent wires, metal layers, and other layout structures. Insufficient spacing can cause short circuits during fabrication.

Minimum Width

Each metal layer must satisfy the minimum width requirements defined by the technology. Wires that are too narrow may increase resistance and affect signal transmission.

Via Size and Spacing

DRC verifies whether the vias used to connect different metal layers meet the required size and spacing rules. Incorrect vias can result in unreliable connections.

Layer Arrangement

The tool checks whether different layout layers are used correctly and according to the technology guidelines.

Overlap and Geometry Checks

DRC identifies unwanted overlaps, missing connections, and other geometric violations that could affect the manufacturability of the design.

Importance of DRC

Design Rule Check is essential because it helps ensure that the layout is physically manufacturable. By detecting layout violations early, DRC reduces the risk of fabrication failures and improves the reliability of the final chip.

Some key benefits of DRC include:

- Detecting manufacturing-related layout errors.
- Ensuring compliance with fabrication rules.
- Improving design reliability and yield.
- Reducing fabrication risks and costs.
- Preparing the design for further verification stages.

A layout that successfully passes DRC is considered compliant with the technology rules and can proceed to additional verification processes.

DRC in the SPI Project

In this project, DRC was performed after the routing stage of the SPI design. The generated physical layout was analysed to verify that all routing and layout structures satisfied the design rules of the selected technology library.

The verification process checked metal widths, spacing requirements, vias, layer geometries, and other physical constraints. Any potential violations would have been reported and corrected before proceeding further.

Project Result

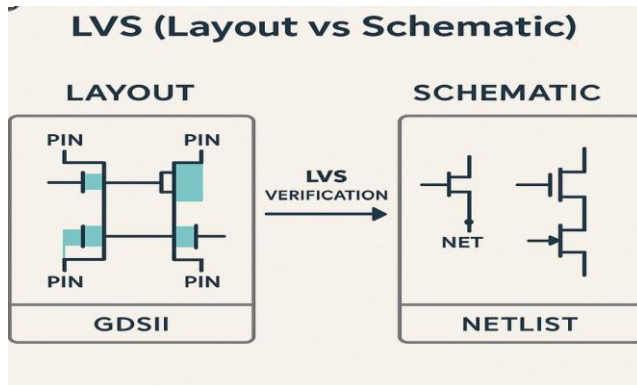
The DRC verification of the SPI layout was completed successfully. The generated layout satisfied all required manufacturing rules and did not contain any design rule violations.

DRC Result: 0 Errors

This result confirmed that the SPI layout was physically correct and suitable for the next verification stage in the ASIC physical design flow. Successfully passing DRC demonstrated that the design could be manufactured according to the selected technology requirements without encountering layout-related fabrication issues.

LVS (Layout Versus Schematic) :

Layout Versus Schematic (LVS) is an important verification step in the ASIC physical design flow. It is performed after the layout has successfully passed the Design Rule Check (DRC) stage. The main objective of LVS is to verify that the final physical layout accurately represents the original circuit design.



During the physical design process, the RTL design is synthesised, placed, and routed to create a physical layout. Although the layout may satisfy all manufacturing rules, it is still necessary to confirm that the implemented layout matches the intended circuit functionality. LVS performs this verification by comparing the extracted netlist from the physical layout with the original schematic or synthesised netlist.

In simple terms, LVS answers an important question:

"Does the physical layout represent the same circuit as the original design?"

If the answer is yes, the design is considered functionally correct and ready for fabrication.

Working of LVS

The LVS tool first extracts the devices and interconnections from the generated layout and creates a layout netlist. This extracted netlist is then compared with the original design netlist generated during synthesis.

The comparison process checks whether both netlists contain the same components and connections. Any mismatch between them indicates that the physical layout does not correctly implement the intended design.

Checks Performed During LVS

The LVS verification process performs several important checks, including:

Component Verification

The tool verifies whether all required components present in the original design are also present in the physical layout.

Connectivity Verification

LVS checks whether all connections between different components are implemented correctly in the layout.

Missing Components Check

The tool identifies any components that are present in the schematic but missing in the physical layout.

Extra Components Check

LVS also detects unnecessary components that may have been introduced during the physical design process.

Netlist Comparison

A complete comparison is performed between the extracted layout netlist and the original design netlist to ensure that both represent the same circuit.

Importance of LVS

LVS is a critical verification step because it ensures the functional correctness of the physical implementation. Even if a layout passes DRC, it may still contain connectivity errors that can affect circuit operation. LVS helps detect such issues before fabrication.

Some important benefits of LVS include:

- Verifies that the layout matches the original design.
- Detects missing or incorrect connections.
- Identifies missing or extra components.
- Prevents functional failures after fabrication.
- Improves overall design reliability and correctness.

Successfully passing LVS gives confidence that the physical layout will perform exactly as intended when manufactured.

LVS in the SPI Project

In this project, LVS verification was performed after the completion of routing and DRC verification. The final SPI layout generated using Qflow was compared against the synthesised gate-level netlist to ensure design correctness.

The LVS tool extracted the devices and connections from the layout and compared them with the original SPI design netlist. The verification process checked all logic components, signal paths, and interconnections to confirm that the physical implementation matched the intended circuit design.

Project Result

The LVS verification for the SPI design was completed successfully. The extracted layout netlist matched the original synthesised netlist without any mismatches.

LVS Result: 0 Errors / 0 Mismatches

This result confirmed that the final physical layout accurately represented the original SPI design. The successful LVS verification demonstrated that all components and connections were implemented correctly, making the design ready for final layout generation and fabrication preparation.

Passing both DRC and LVS verification indicated that the SPI design was physically correct, functionally accurate, and suitable for manufacturing.

GDS (Graphic Data System) :

GDS (Graphic Data System) is the final output generated at the end of the ASIC physical design flow. It is the industry-standard file format used to represent the complete physical layout of an integrated circuit. After all design implementation and verification stages are successfully completed, the layout is converted into a GDS file, which serves as the final design database for chip fabrication.

The GDS file contains all the geometric and physical information required to manufacture the integrated circuit on a silicon wafer. It provides a detailed representation of the chip layout, including the placement of components, routing structures, metal layers, vias, and other physical features of the design.

Since the GDS file is the direct input to the semiconductor fabrication process, it is considered one of the most important deliverables in the ASIC design flow. Any errors present in the layout before GDS generation can result in manufacturing defects, making verification stages such as DRC and LVS extremely important.

Information Contained in a GDS File

A GDS file stores complete physical layout information, including:

Layout Layers

The file contains all the layers used in the design, such as diffusion layers, polysilicon layers, contact layers, vias, and metal routing layers.

Component Locations

The exact positions of all standard cells and other layout components are stored in the GDS file.

Metal Interconnections

All signal routing paths and power connections created during the routing stage are included in the layout database.

Physical Dimensions

The GDS file contains accurate geometric information such as widths, lengths, spacing, and overall chip dimensions required for fabrication.

Design Hierarchy

The hierarchical structure of the design is preserved, allowing efficient representation of large and complex integrated circuits.

Importance of GDS Generation

GDS generation is the final step that transforms the verified layout into a format that can be understood by semiconductor fabrication facilities. It acts as the bridge between chip design and chip manufacturing.

Some key advantages of GDS generation include:

- Provides a complete physical representation of the design.
- Serves as the final input for fabrication.
- Preserves all layout and connectivity information.
- Enables accurate manufacturing of the integrated circuit.
- Marks the successful completion of the ASIC physical design flow.

A correctly generated GDS file indicates that the design has successfully passed implementation and verification stages and is ready for fabrication.

GDS Generation in the SPI Project

In this project, the GDS file was generated after completing all major physical design and verification stages, including synthesis, placement, routing, Static Timing Analysis (STA), Design Rule Check (DRC), and Layout Versus Schematic (LVS) verification.

Since the SPI design successfully passed both DRC and LVS checks, the final verified layout was converted into the GDS format. The generated GDS file contained all layout information, including standard cell placement, routing structures, metal layers, vias, and physical dimensions of the SPI controller.

The GDS generation stage marked the completion of the ASIC implementation process for the SPI design using the Qflow environment.

Project Result

The GDS file for the SPI design was generated successfully after completing all physical design and verification stages. The final layout was converted into a fabrication-ready format without any errors.

GDS Generation Status: Successful

The generated GDS file represents the complete physical implementation of the SPI controller and serves as the final output of the ASIC physical design flow. This confirmed that the design was successfully transformed from a Verilog RTL description into a manufacturable chip layout.

The successful generation of the GDS file marked the final milestone of the project and demonstrated the complete implementation of the SPI design using the Qflow ASIC physical design flow.

Results and Discussion :

The physical design implementation of the SPI Top module was completed using the Qflow ASIC design flow with the OSU018 technology library. The complete flow included synthesis, placement, routing, static timing analysis (STA), Design Rule Check (DRC), Layout Versus Schematic (LVS), and GDSII generation.

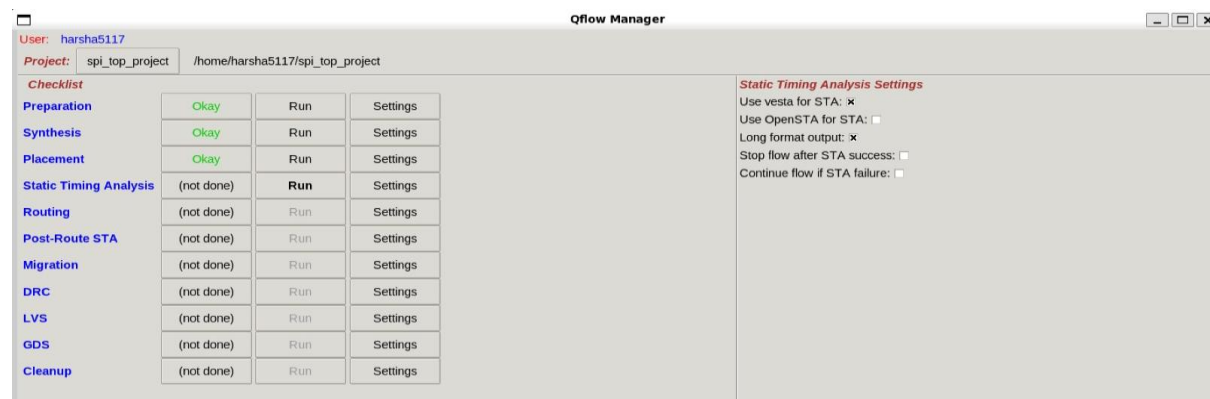
Synthesis Result

The RTL description of the SPI design was synthesised into a gate-level netlist using the OSU018 standard cell library. The synthesis process successfully converted the Verilog design into optimised standard cell representations.

The synthesised design contained:

- Standard cell based implementation
- Flip-flops: DFFSR cells
- Combinational logic gates such as NAND, NOR, AOI, OAI, XOR, buffers and inverters

The generated netlist was used as input for further physical design steps.



Placement and Routing Result

After synthesis, placement was performed to arrange standard cells inside the available chip area. Routing was then completed to establish physical connections between cells.

The routing process completed successfully with:

- Total routes completed: **22209**
- Failed routes: **0**

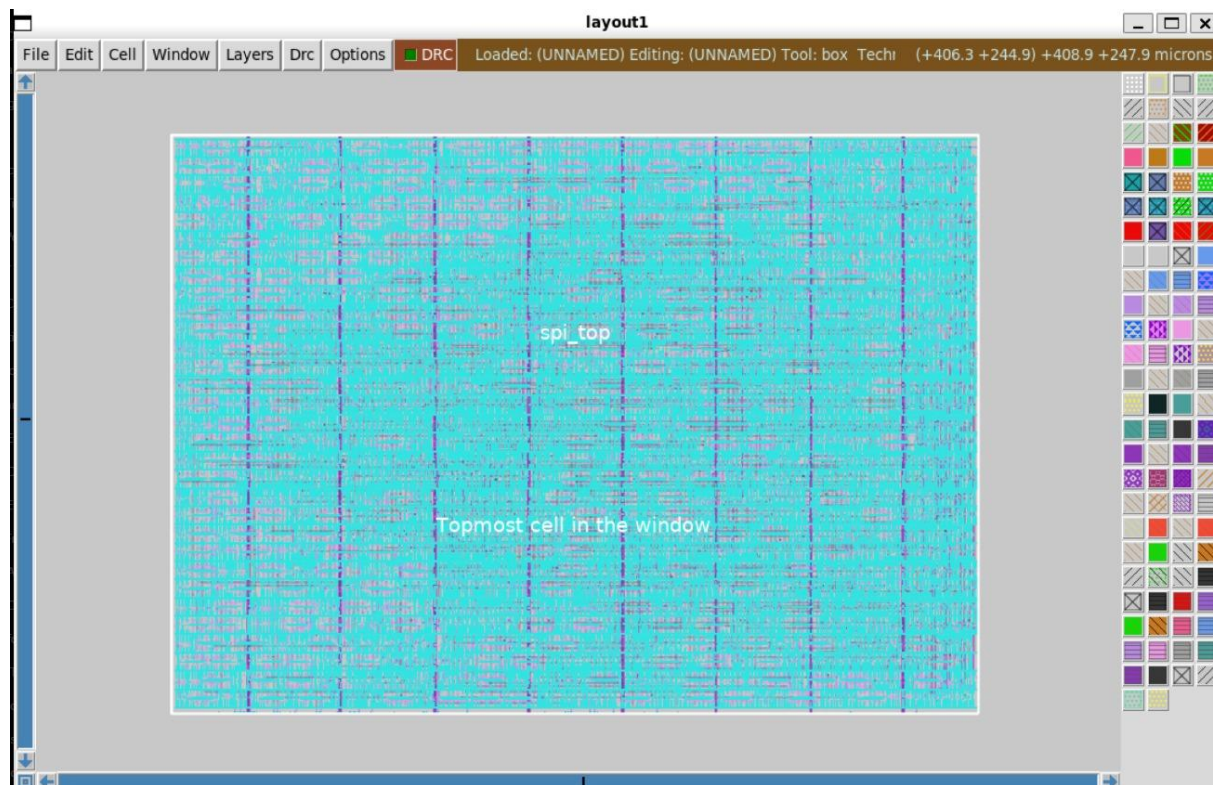
This indicates that all required connections were successfully routed without congestion issues.

```
Nets remaining: 2300
Nets remaining: 2200
Nets remaining: 2100
Nets remaining: 2000
Nets remaining: 1900
Nets remaining: 1800
Nets remaining: 1700
Nets remaining: 1600
Nets remaining: 1500
Nets remaining: 1400
Nets remaining: 1200
Nets remaining: 1100
Nets remaining: 1000
Nets remaining: 800
Nets remaining: 700
Nets remaining: 600
Nets remaining: 500
Nets remaining: 400
Nets remaining: 90
Nets remaining: 80
Nets remaining: 70
Nets remaining: 60
Nets remaining: 50
Nets remaining: 40
Nets remaining: 20
Progress: Stage 3 total routes completed: 22209
No failed routes!
*** Writing DEF file spi_top_route.def
Final: No failed routes!
*** Writing RC file spi_top_route.rc
Running annotate.tcl antenna.out /home/harsha5117/spi_top_project/spi_top.rtl.nopwr.v
/home/harsha5117/spi_top_project/spi_top.spc /home/harsha5117/spi_top_project/spi_top.rtl.nopwr.anno.v
/home/harsha5117/spi_top_project/spi_top.anno.spc /usr/share/qflow/tech/osu018/osu018_stdcells.sp /home/harsha5117/spi_top_project/spi_top_powerground
Running annotate.tcl
found cell FILL pinlist vdd gnd
Done with annotate.tcl
harsha5117@Ursharsha:~/spi_top_project$ qflow -T osu018 route spi_top
```

Layout Generation (GDSII)

The final physical layout of the SPI design was generated using Magic. The layout contains standard cells, power connections, and routing layers based on OSU018 fabrication rules.

The generated layout represents the final ASIC physical implementation that can be used for fabrication.



Static Timing Analysis (STA)

Static Timing Analysis was performed to verify the timing performance of the implemented design.

The important timing results are:

- Total paths analyzed: **321**
- Worst setup delay: **4.161 ns**
- Maximum operating frequency: **240.33 MHz**
- Critical path: **DFFSR_42 → DFFSR_240**

The critical path delay is mainly due to a long combinational logic chain containing NAND, NOR, AOI, and OAI cells.

Hold analysis was also performed:

- Minimum delay: **267 ps**
- No hold violations detected

Hence, the design is setup-limited and satisfies timing requirements.

```

Parsing library "osu018_stdcells"
End of library at line 6141
Parsing module "spi_top"
Lib read /usr/share/qflow/tech/osu018/osu018_stdcells.lib: Processed 6142 lines.
Verilog netlist read: Processed 4753 lines.
Number of paths analyzed: 321

```

```

Top 20 maximum delay paths:
Path DFFSR_42/CLK to DFFSR_240/D delay 4160.96 ps
    0.0 ps  wb_clk_i_bF_buf3_bF_buf2:  CLKBUF1_26/Y ->  DFFSR_42/CLK
    642.6 ps          rx_negedge:      DFFSR_42/Q ->  NAND3X1_253/C
    1016.1 ps          _1958_:  NAND3X1_253/Y ->  NOR3X1_5/C
    1213.3 ps          _1985_:  NOR3X1_5/Y ->  NAND2X1_118/B
    1320.0 ps          _1994_:  NAND2X1_118/Y ->  NAND3X1_263/B
    1549.9 ps          _1995_:  NAND3X1_263/Y ->  NAND2X1_120/A
    1753.7 ps          _1999_:  NAND2X1_120/Y ->  NOR3X1_7/C
    1851.4 ps          _2000_:  NOR3X1_7/Y ->  AOI21X1_66/B

```

DRC Verification

Design Rule Check was performed using Magic to ensure that the generated layout follows the fabrication technology rules.

The DRC result obtained:

- DRC errors: 0

This confirms that the layout satisfies the required physical design rules.

```

Technology set to osu018
Regenerating files for existing project spi_top
Warning: No LVS or post-route STA logfiles found.
Running magic 8.3.105
magic -dnull -noconsole /home/harsha5117/spi_top_project/run_drc_spi_top.tcl

Magic 8.3 revision 105 - Compiled on Mon, 06 Dec 2021 22:32:27 +0200.
Starting magic under Tcl interpreter
Using the terminal as the console.
Using NULL graphics device.
Processing system .magicrc file
Input style lambda=0.10(p): scaleFactor=10, multiplier=1
Contact size value ignored (using GDS generation rules).
Contact size value ignored (using GDS generation rules).
Contact size value ignored (using GDS generation rules).
Contact size value ignored (using GDS generation rules).
Contact size value ignored (using GDS generation rules).
Contact size value ignored (using GDS generation rules).
Loading "/home/harsha5117/spi_top_project/run_drc_spi_top.tcl" from command line.
Reading LEF data from file /usr/share/qflow/tech/osu018/osu018_stdcells.lef.
This action cannot be undone.
LEF read, Line 16 (Message): Unknown keyword "OBS" in LEF file; ignoring
LEF read, Line 17 (Message): Unknown keyword "PIN" in LEF file; ignoring
LEF read: Processed 2941 lines.
File spi_top.mag couldn't be read
No such file or directory
Creating new cell
drc = 0
harsha5117@Ursharsha:~/spi_top_project$ qflow -T osu018 gdsii spi_top

-----
Qflow project setup

```

LVS Verification

Layout Versus Schematic verification was performed to check whether the final layout matches the synthesized netlist.

Results:

- Devices matched: **3103**
- Nets matched: **3150**
- Net, device, pin, and property mismatches: **0**
- Total errors: **0**

Therefore, the physical layout is electrically equivalent to the original design.

```
Class: INVX8                      instances: 26
Circuit contains 3150 nets, and 2 disconnected pins.
Contents of circuit 2: Circuit: 'spi_top'
Circuit spi_top contains 3103 device instances.
Class: NOR3X1                     instances: 9
Class: OR2X2                     instances: 13
Class: MUX2X1                     instances: 48
Class: AOI22X1                   instances: 90
Class: NOR2X1                     instances: 263
Class: NAND3X1                   instances: 276
Class: OAI22X1                   instances: 123
Class: XOR2X1                     instances: 6
Class: AOI21X1                   instances: 224
Class: NAND2X1                   instances: 262
Class: OAI21X1                   instances: 711
Class: XNOR2X1                   instances: 14
Class: BUF2X1                     instances: 82
Class: BUF4X1                     instances: 277
Class: DFFSR                      instances: 253
Class: AND2X2                     instances: 29
Class: CLKBUFF1                  instances: 67
Class: INVX1                      instances: 158
Class: INVX2                      instances: 160
Class: INVX4                      instances: 12
Class: INVX8                      instances: 26
Circuit contains 3150 nets, and 2 disconnected pins.

Circuit 1 contains 3103 devices, Circuit 2 contains 3103 devices.
Circuit 1 contains 3150 nets, Circuit 2 contains 3150 nets.

Netlists match uniquely.
Result: Circuits match uniquely.
Logging to file "comp.out" disabled
LVS Done.
Running count_lvs.py
/usr/lib/qflow/scripts/count_lvs.py
LVS reports no net, device, pin, or property mismatches.

Total errors = 0
```

Overall Discussion :

Checklist			
Preparation	Okay	Run	Settings
Synthesis	Okay	Run	Settings
Placement	Okay	Run	Settings
Static Timing Analysis	Okay	Run	Settings
Routing	Okay	Run	Settings
Post-Route STA	Okay	Run	Settings
Migration	Okay	Run	Settings
DRC	Okay	Run	Settings
LVS	Okay	Run	Settings
GDS	Okay	Run	Settings
Cleanup	Okay	Run	Settings

Current qflow status: Next project action is clean
Current qflow status: Project is completed.
Displaying the completed routed layout, full view

The SPI design successfully completed the complete ASIC physical design flow. The final routed layout passed DRC and LVS verification, confirming correct physical implementation. STA analysis showed a maximum operating frequency of approximately **240 MHz**, demonstrating that the implemented design meets timing requirements.

The successful generation of the GDSII file indicates that the SPI design is ready for further fabrication-level processing.

Conclusion :

This internship project focused on the implementation of a Serial Peripheral Interface (SPI) design using the complete Physical design flow in the Qflow open-source VLSI design environment. The project provided practical exposure to the various stages involved in transforming a digital RTL design into a physical chip layout suitable for semiconductor fabrication.

The SPI design, written in Verilog HDL, was successfully taken through all major stages of the ASIC backend flow, including logic synthesis, placement, routing, static timing analysis, Design Rule Check (DRC), Layout Versus Schematic (LVS) verification, and GDS generation. Each stage played an important role in converting the logical design into a physical implementation while ensuring performance, correctness, and manufacturability.

Through the synthesis process, the RTL design was converted into an optimized gate-level netlist using standard cells. The placement and routing stages created the physical structure and interconnections required for circuit implementation. Static Timing Analysis was performed to evaluate the timing performance of the design and verify that timing requirements were satisfied. Physical verification using DRC and LVS confirmed that the generated layout followed all manufacturing rules and accurately matched the original design. Finally, the verified layout was successfully converted into a GDS file, representing the final fabrication-ready output.

One of the major achievements of this project was the successful completion of physical verification with zero DRC errors and zero LVS mismatches, indicating that the design was both physically correct and functionally accurate. The project also provided a clear understanding of how different stages of the ASIC physical design flow contribute to the development of reliable and efficient integrated circuits.

Overall, this internship served as a valuable learning experience in the field of VLSI and semiconductor design. It helped strengthen the understanding of backend ASIC design concepts, physical verification techniques, timing analysis, and layout implementation. The knowledge and practical skills gained through this project provide a strong foundation for future learning and career opportunities in VLSI design and the semiconductor industry.

Learning Outcome :

- Gained practical knowledge of the VLSI Physical Design flow from RTL design to final layout generation.
- Learned the important stages of physical design including synthesis, placement, routing, STA, DRC, and LVS verification.
- Developed hands-on experience with Qflow and VLSI EDA tools in a Linux environment.
- Understood concepts such as standard cell placement, routing, timing analysis, and layout verification.
- Learned to analyze timing reports, critical paths, delay, and design performance using STA.
- Gained experience in debugging design flow issues and successfully generating the final GDSII layout.
- Understood the importance of DRC and LVS checks for ensuring correct and error-free physical implementation.